北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Liu	**名 First Name**	Weiliang		
BUPT 学号 BUPT number	2022213200	**QM 学号 QM number**	221166552	**班级 Class**	2022215107
论文题目 Project Title	Android Chinese words game app				

是否完成任务书中所定的中期目标？Targets met (as set in the Specification)?
[YES]

已完成工作 Finished work:

1. Technology Selection and Platform Migration

The original Specification planned to use Unity Engine with C# as the primary development platform. During early-stage development, it became evident that native Android development (Java + XML) offered significant advantages for this project.

The final technology stack is:

- Platform: Android (Compile SDK 36)
- Language: Java 11 + XML
- IDE: Android Studio
- Database: SQLite
- Key Libraries: HanLP (1.8.6), pinyin4j (2.5.0), Gson (2.8.9), iFlyTek SparkChain SDK

2. User Interface Framework Implementation

A complete user interface framework has been implemented with 7 Activity screens and 9 XML layout files:

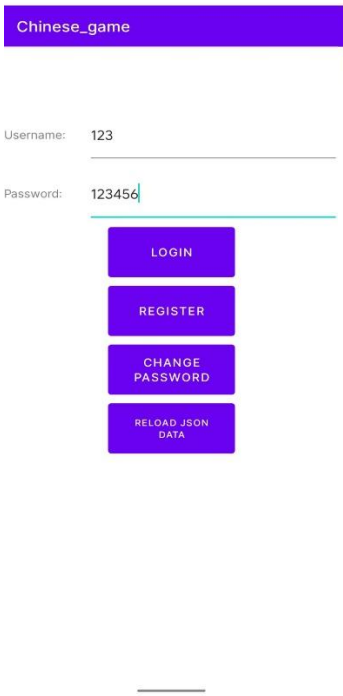
Activity Page	Layout File	Function Purpose
Main Activity	activity_main.xml	Login interface, supports username/password input, includes buttons for login, registration, password modification, and JSON data reloading
Registration Page	activity_register.xml	User registration form page
Password Modification Page	activity_update_test.xml	Password modification form page (requires original password verification and new password input)
Game Selection Page	activity_game_choice.xml	Main game selection interface with toolbar menu, difficulty selection dialog, and loading indicator
Game Main Page	activity_game.xml	Main game container supporting multiple game modes
User Profile Page	activity_user_profile.xml	User profile viewing and editing interface (supports username/email modification and game data statistics viewing)
User Achievements Page	activity_user_achievements.xml	Scrollable list for displaying user achievements

Additional UI components:

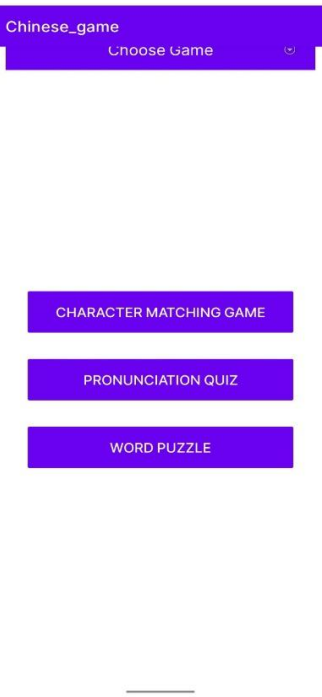
- dialog_select_difficulty.xml— Difficulty selection dialog (Easy / Medium / Hard)
- dialog_loading_questions.xml — Loading progress indicator during question preparation
- VoiceWaveView — Custom view for real-time voice volume visualization during recording

Navigation Flow:

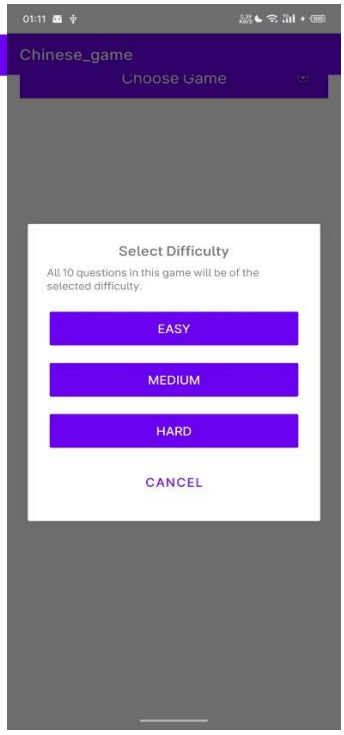
```
MainActivity (Login)
├── register (Registration)
├── update_test (Change Password)
└── game_choice (Game Selection Hub)
    ├── GameActivity — CHARACTER_MATCHING
    ├── GameActivity — PRONUNCIATION_QUIZ
    ├── GameActivity — WORD_PUZZLE (placeholder)
    ├── UserProfileActivity (via menu)
    └── UserAchievementsActivity (via menu)
```



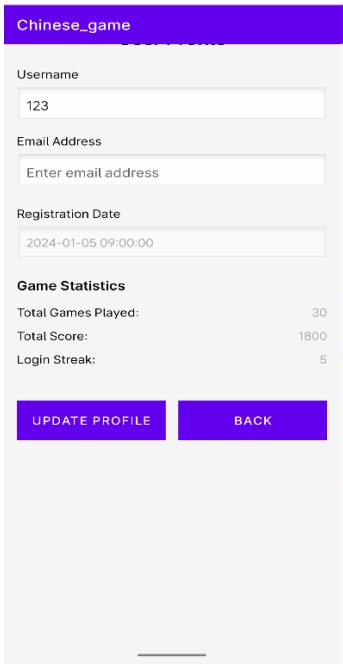
login screen



game selection screen



difficulty selection



user profile

character matching game

pronunciation quiz

3. Database Design and Implementation

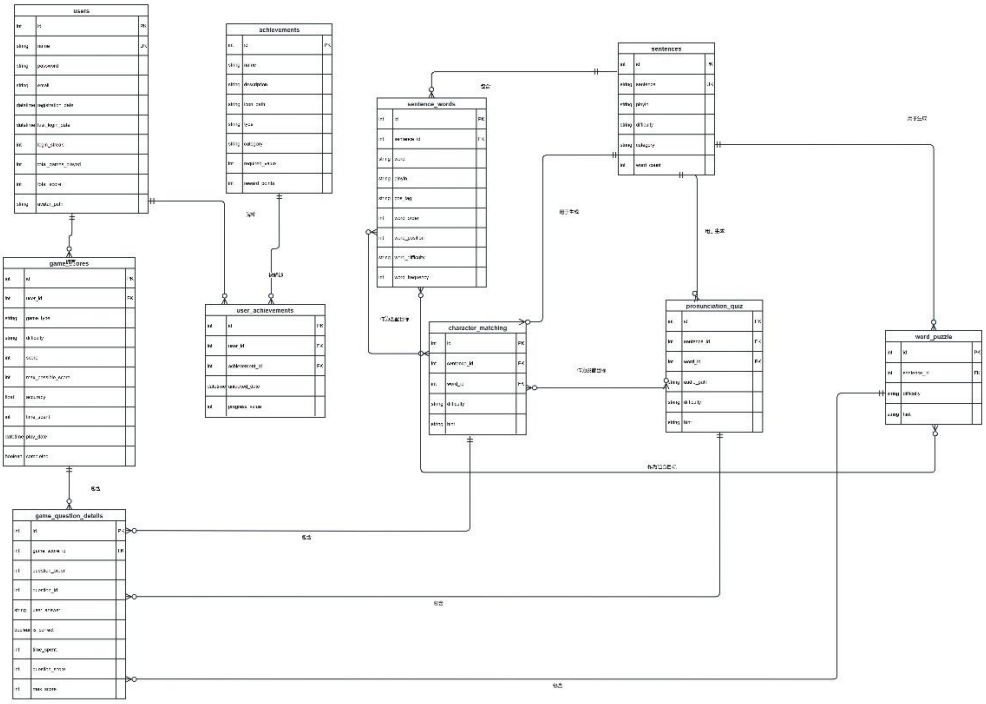
A comprehensive SQLite database (**ChineseGame.db**) has been designed and implemented using a custom **MYsqliteopenhelper.class**. The database architecture includes:

DAO Layer (10 Data Access Objects):

Data Access Object	Corresponding Model Class	Core Operations
UserDAO	User	User registration, login, profile modification, login information tracking
AchievementDAO	Achievement, UserAchievement	Achievement definition, user achievement progress tracking, achievement badge unlocking
GameScoreDAO	GameScore	Game score recording (per game round), historical score query
GameQuestionDetailDAO	GameQuestionDetail	Storage of answer status and time consumption for each question
CharacterPinyinMatchingDAO	CharacterPinyinMatching	Management of character-pinyin matching game question bank
PronunciationQuizDAO	PronunciationQuiz	Management of pronunciation quiz game metadata
SentenceWordDAO	SentenceWord	Storage of game vocabulary and distractor vocabulary
WordPuzzleDAO	WordPuzzle	Preparation for word puzzle game development
GameGeneralDAO	GameGeneralData	Access and management of general game data

Design Patterns:

- DaoFactory — Singleton pattern for consistent DAO instance management across the application
- DataManager — Centralized JSON-to-SQLite data pipeline handling table reset, JSON parsing, and bulk insert
- SeedDataLoader — Specialized loader for sentences and sentence words, integrating HanLP segmentation for word position updates



Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(32)	UNIQUE, NOT NULL
password	VARCHAR(32)	NOT NULL
email	VARCHAR(64)	—
registration_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
last_login_date	DATETIME	—
login_streak	INTEGER	DEFAULT 0
total_games_played	INTEGER	DEFAULT 0
total_score	INTEGER	DEFAULT 0
avatar_path	VARCHAR(255)	—

Users

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
name	VARCHAR(64)	NOT NULL
description	TEXT	—
icon_path	VARCHAR(255)	—
type	VARCHAR(32)	NOT NULL
category	VARCHAR(32)	NOT NULL
required_value	INTEGER	DEFAULT 0
reward_points	INTEGER	DEFAULT 0

achievements

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
user_id	INTEGER	NOT NULL, FK → users(id)
game_type	VARCHAR(32)	NOT NULL
difficulty	VARCHAR(16)	NOT NULL
score	INTEGER	NOT NULL
max_possible_score	INTEGER	DEFAULT 0
accuracy	REAL	DEFAULT 0.0
time_spent	INTEGER	DEFAULT 0
play_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
completed	INTEGER	DEFAULT 1

game_scores

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
user_id	INTEGER	NOT NULL, FK → users(id)
achievement_id	INTEGER	NOT NULL, FK → achievements(id)
unlocked_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
progress_value	INTEGER	DEFAULT 0

user_achievements

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
game_score_id	INTEGER	NOT NULL, FK → game_scores(id)
question_order	INTEGER	NOT NULL
question_id	INTEGER	NOT NULL
user_answer	TEXT	—
is_correct	INTEGER	DEFAULT 0
time_spent	INTEGER	DEFAULT 0
question_score	INTEGER	DEFAULT 0
max_score	INTEGER	DEFAULT 10

game_question_

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
sentence	TEXT	NOT NULL, UNIQUE
pinyin	TEXT	—
difficulty	VARCHAR(16)	NOT NULL, DEFAULT 'EASY'
category	VARCHAR(32)	—
word_count	INTEGER	DEFAULT 0

details sentences

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT

sentence_id	INTEGER	NOT NULL, FK → sentences(id) ON DELETE CASCADE
word	TEXT	NOT NULL
pinyin	TEXT	—
pos_tag	VARCHAR(8)	NOT NULL
word_order	INTEGER	NOT NULL
word_position	INTEGER	DEFAULT 0
word_difficulty	VARCHAR(16)	DEFAULT 'EASY'
word_frequency	INTEGER	DEFAULT 0

sentence_words

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
Sentence_id	INTEGER	NOT NULL, FK → sentences(id)
word_id	INTEGER	NOT NULL, FK → sentence_words(id)
difficulty	VARCHAR(16)	NOT NULL
hint	TEXT	—

character_matchi

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
sentence_id	INTEGER	NOT NULL, FK → sentences(id)
word_id	INTEGER	NOT NULL, FK → sentence_words(id)
audio_path	VARCHAR(255)	—
difficulty	VARCHAR(16)	NOT NULL
hint	TEXT	—

pronunciation_quiz

Column	Type	Constraints
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
sentence_id	INTEGER	NOT NULL, FK → sentences(id)
difficulty	VARCHAR(16)	NOT NULL
hint	TEXT	—

word_puzzle

```java
public List<Achievement> checkAndUnlockMilestoneAchievements(int userId) {
    List<Achievement> newlyUnlocked = new ArrayList<>();
    User user = userDao.findById(userId);
    if (user == null) return newlyUnlocked;

    List<Achievement> milestones = findByType(Achievement.AchievementType.MILESTONE);

    for (Achievement achievement : milestones) {
        if (isAchievementUnlocked(userId, achievement.getId())) {
            continue; // 已经解锁
        }

        boolean shouldUnlock = false;

        switch (achievement.getCategory()) {
            case GENERAL:
                if (achievement.getName().equals("First Steps")) {
                    shouldUnlock = user.getTotalGamesPlayed() >= achievement.getRequiredValue();
                }
                break;
            case LOGIN_STREAK:
                if (achievement.getName().equals("Login Streak")) {
                    shouldUnlock = user.getLoginStreak() >= achievement.getRequiredValue();
                }
                break;
        }

        if (shouldUnlock) {
            UserAchievement ua = new UserAchievement(userId, achievement.getId());
            if (unlockAchievement(ua)) {
                newlyUnlocked.add(achievement);
            }
        }
    }
```

4. JSON Question Bank and Data Loading

A rich question bank has been built using 7 JSON data files stored in **assets/json/**:

File Name	Content	Scale
sentences.json	Sentence bank with pinyin, difficulty labels and categories	3794 lines
sentence_words.json	Specialized vocabulary bank for sentence-based games	Loaded by InitialDataLoader
character_matching.json	Question bank for character-pinyin matching game	Supports multiple difficulty levels
pronunciation_quiz.json	Target question bank for pronunciation quiz game	Supports multiple difficulty levels
word_puzzle.json	Question bank for word puzzle game	Data preparation completed
achievements.json	Achievement definition file	Includes milestone achievements and game-specific achievements
users.json, game_scores.json, user_achievements.json	Initial test data	—

Data Loading Pipeline:

1. User triggers "Reload JSON Data" or app initializes
2. **DataManager.resetAllTablesAndSequences()** clears all tables
3. JSON files are parsed with Gson and bulk-inserted into SQLite
4. SeedDataLoader processes sentences: imports text, runs HanLP segmentation, updates word positions

JSON Data Structure Examples

1.1 users.json — 用户数据

```
[
  {
    "name": "admin",                // 用户名 (UNIQUE)
    "password": "admin123",         // 密码
    "email": "admin@chinesegame.com",
    "registration_date": "2024-01-01 00:00:00",
    "last_login_date": "2024-01-20 10:00:00",
    "login_streak": 7,              // 连续登录天数
    "total_games_played": 50,       // 总游戏次数
    "total_score": 2500,            // 总分
    "avatar_path": null
  }
]
```

1.2 achievements.json — 成就定义

```
[
  {
    "name": "First Steps",                  // 成就名
    "description": "完成10个游戏",            // 描述
    "icon_path": "/icons/first_steps.png",
    "type": "MILESTONE",                    // 类型: MILESTONE / GAME_SPECIFIC
    "category": "GENERAL",                  // 类别: GENERAL / LOGIN_STREAK /
    "required_value": 10,                   // 达成所需值
    "reward_points": 50                     // 奖励积分
  },
  {
    "name": "Perfect Match",
    "description": "在Character Matching中连续10次正确",
    "type": "GAME_SPECIFIC",
    "category": "CHARACTER_MATCHING",
    "required_value": 10,
    "reward_points": 75
  }
]
```

1.3 user_achievements.json — 用户成就记录

```
[
  {
    "user_id": 1,                           // → users.id
    "achievement_id": 1,                    // → achievements.id
    "unlocked_date": "2024-01-15 10:00:00", // 解锁时间
    "progress_value": 0                     // 进度值
  }
]
```

1.4 game_scores.json — 游戏成绩

```
[
  {
    "user_id": 1,                           // → users.id
    "game_type": "CHARACTER_MATCHING",      // 游戏类型
    "difficulty": "EASY",                   // 难度: EASY / MEDIUM / HARD
    "score": 95,                            // 得分
    "max_possible_score": 100,              // 满分
    "accuracy": 0.95,                       // 正确率
    "time_spent": 45000,                    // 耗时 (ms)
    "play_date": "2024-01-20 10:30:00",
    "completed": true                       // 是否完成
  }
]
```

1.5 sentences.json — 句子题库 (3794 lines)

```
[
  {
    "id": 1,
    "sentence": "我喜欢在北京学习中文",                    // 中文句子
    "pinyin": "wǒ xǐ huān zài běi jīng xué xí zhōng wén", // 拼音
    "difficulty": "MEDIUM",                               // 难度
    "category": "日常学习",                                // 分类
    "word_count": 6                                       // 词数
  },
  {
    "id": 2,
    "sentence": "今天天气很好",
    "pinyin": "jīn tiān tiān qì hěn hǎo",
    "difficulty": "EASY",
    "category": "日常生活",
    "word_count": 3
  },
  {
```

5. Character-Pinyin Matching Game

The Character-Pinyin Matching game has been fully implemented with the following features:

- Core Gameplay: Display a Pinyin prompt; user selects the correct Chinese word from 4 options

- Session Structure: 10 questions per session with visual progress tracking (x/10)
- Hint System: Optional hints available for each question to assist learners
- Randomized Selection: Questions and distractors are randomly selected from the database, ensuring variety across sessions
- Three Difficulty Levels: Easy (common characters), Medium (intermediate vocabulary), Hard (advanced words) — selected before game start

Technical Implementation:

- CharacterMatchingDao queries the question bank with difficulty filtering
- Randomized distractor generation ensures wrong options are plausible but distinct
- Real-time result feedback with visual indicators (correct/incorrect)
- Game session data (score, per-question details) saved to database upon completion

getRandomSentenceWordsForGame()

```java
@Override
public List<CharacterMatching> getRandomSentenceWordsForGame(int count, String difficulty) {
    if (count <= 0) return new ArrayList<>();
    SQLiteDatabase db = dbHelper.getPersistentDatabase();
    String sql = "SELECT sw.id AS word_id, sw.sentence_id, sw.word, sw.pinyin, sw.pos_tag, s.sentence, s.difficulty " +
            "FROM sentence_words sw JOIN sentences s ON sw.sentence_id = s.id " +
            "WHERE sw.pos_tag NOT IN " + EXCLUDED_POS_FOR_GAME;
    List<String> argsList = new ArrayList<>();
    if (difficulty != null && !difficulty.isEmpty()) {
        sql += " AND sw.word_difficulty = ? ";
        argsList.add(difficulty);
    }
    sql += " ORDER BY RANDOM() LIMIT ?";
    argsList.add(String.valueOf(Math.max(count * 3, b: 30)));
    Cursor cursor = db.rawQuery(sql, argsList.toArray(new String[0]));
    List<CharacterMatching> list = new ArrayList<>();
    while (cursor.moveToNext()) {
        CharacterMatching q = cursorToCharacterMatching(cursor);
        String w = q.getWord();
        if (w != null && !w.trim().isEmpty() && !ONLY_DIGITS_PUNCT.matcher(w.trim()).matches()) {
            list.add(q);
            if (list.size() >= count) break;
        }
    }
    cursor.close();
    return list;
}
```

getRandomWordsByPosTag()

```java
@Override
public List<SentenceWord> getRandomWordsByPosTag(String posTag, int excludeWordId, int count) {
    if (posTag == null || count <= 0) return new ArrayList<>();

    SQLiteDatabase db = dbHelper.getPersistentDatabase();
    String[] args = new String[]{posTag, String.valueOf(excludeWordId)};
    Cursor cursor = db.rawQuery(
        "SELECT id, sentence_id, word, pinyin, pos_tag, word_order " +
        "FROM sentence_words WHERE pos_tag = ? AND id != ?",
        args);

    List<SentenceWord> list = new ArrayList<>();
    while (cursor.moveToNext()) {
        list.add(cursorToSentenceWord(cursor));
    }
    cursor.close();

    if (list.size() <= count) return list;
    Collections.shuffle(list);
    return list.subList(0, count);
}
```

6. Pronunciation Quiz with iFlyTek ISE

The Pronunciation Quiz has been implemented with professional-grade speech evaluation powered by iFlyTek's Intelligent Speech Evaluation (ISE) engine:

a) Core Gameplay: Display a target word/sentence with Pinyin; user holds the microphone button to record their pronunciation; receive instant multi-dimensional feedback

b) Speech Evaluation Engine: IseEvaluator.class manages the complete ISE WebSocket lifecycle:

- WebSocket connection establishment with iFlyTek API authentication
- Audio stream chunking and transmission
- XML result parsing for multi-dimensional scores

c) Score Mapping: Raw ISE score (0–100) is normalized to a game score (0–10) for consistent user experience

d) UI Features:

- Hold-to-record microphone button
- Real-time VoiceWaveView volume visualization during recording
- Detailed ISE feedback display after evaluation
- Retry mechanism for re-attempting pronunciation

startAudioCapture()

```java
private void startAudioCapture(WebSocket ws) {
    try {
        int bufSize = Math.max(
                AudioRecord.getMinBufferSize(SAMPLE_RATE,
                        AudioFormat.CHANNEL_IN_MONO, AudioFormat.ENCODING_PCM_16BIT),
                SAMPLE_RATE * 2) * 2;

        audioRecord = new AudioRecord(MediaRecorder.AudioSource.MIC,
                SAMPLE_RATE, AudioFormat.CHANNEL_IN_MONO,
                AudioFormat.ENCODING_PCM_16BIT, bufSize);

        if (audioRecord.getState() != AudioRecord.STATE_INITIALIZED) {
            notifyError(msg: "麦克风初始化失败");
            return;
        }

        audioRecord.startRecording();
        isRecording.set(newValue: true);

        new Thread(() -> {
            byte[] buffer = new byte[FRAME_SIZE];
            int frameIndex = 0;

            while (isRecording.get()) {
                int read = audioRecord.read(buffer, 0, FRAME_SIZE);
                if (read <= 0) continue;

                try {
                    int status;
                    if (frameIndex == 0) {
                        status = 0; // first frame
                    } else {
                        status = 1; // middle frame
                    }
                    sendFrame(ws, buffer, read, status, frameIndex == 0 ? targetText : null);
                    frameIndex++;

                    if (rmsCallback != null) {
                        float rms = calcRms(buffer, read);
                        mainHandler.post(() -> rmsCallback.onRmsChanged(rms));
                    }
                    Thread.sleep(millis: 40);
                } catch (Exception e) {
                    Log.e(TAG, "发送帧异常", e);
                    break;
                }
            }

            // 发送最后一帧
            try {
                sendFrame(ws, new byte[0], len: 0, status: 2, text: null);
            } catch (Exception e) {
                Log.e(TAG, "发送最后帧失败", e);
            }

            isSendingComplete.set(newValue: true);
            releaseAudioRecord();
            Log.d(TAG, "音频发送完毕, 共" + frameIndex + "帧");
        }, name: "ISE-Audio").start();

    } catch (Exception e) {
        Log.e(TAG, "启动录音失败", e);
        notifyError("启动录音失败: " + e.getMessage());
    }
}
```

calcRms()

```java
    }

    private float calcRms(byte[] buf, int len) {
        long sum = 0;
        for (int i = 0; i + 1 < len; i += 2) {
            short s = (short) ((buf[i + 1] << 8) | (buf[i] & 0xFF));
            sum += (long) s * s;
        }
        double rms = Math.sqrt((double) sum / (len / 2));
        return (float) (20 * Math.log10(rms + 1));
    }

    private void notifyError(String msg) {
        mainHandler.post(() -> {
            if (callback != null) callback.onError(msg);
        });
    }
```

parseAndNotifyResult()

```java
// ================== 结果解析 ==================

private void parseAndNotifyResult(String rawResult) {
    // 去掉XML前面可能的乱码（base64解码残留）
    int xmlStart = rawResult.indexOf(str: "<?xml");
    if (xmlStart < 0) xmlStart = rawResult.indexOf(str: "<xml_result");
    String xml = xmlStart > 0 ? rawResult.substring(xmlStart) : rawResult;

    Log.i(TAG, "ISE原始XML: " + xml);

    try {
        // 讯飞ISE read_word 的评分在 <read_word> 元素的 **属性** 中
        // 例: <read_word total_score="85.5" fluency_score="80.2" ...>
        float totalScore = extractAttr(xml, tagName: "read_word", attrName: "total_score");
        float phonScore  = extractAttr(xml, tagName: "read_word", attrName: "phone_score");
        float fluency    = extractAttr(xml, tagName: "read_word", attrName: "fluency_score");
        float integrity  = extractAttr(xml, tagName: "read_word", attrName: "integrity_score");
        float toneScore  = extractAttr(xml, tagName: "read_word", attrName: "tone_score");

        // content 也在属性里: <read_word content="你好">
        String content = extractAttrStr(xml, tagName: "read_word", attrName: "content");
        if (content.isEmpty()) {
            content = extractAttrStr(xml, tagName: "sentence", attrName: "content");
        }

        // 用 phone_score 作为准确度（read_word 模式无 accuracy_score）
        float accuracy = phonScore;

        Log.d(TAG, "评测解析: total=" + totalScore + " phone=" + phonScore
                + " fluency=" + fluency + " integrity=" + integrity
                + " tone=" + toneScore + " content=" + content);

        EvalResult result = new EvalResult(content, totalScore, accuracy, fluency, integrity, xml);
        mainHandler.post(() -> {
            if (callback != null) callback.onResult(result);
        });
    } catch (Exception e) {
        Log.e(TAG, "解析XML失败", e);
        EvalResult fallback = new EvalResult(recognizedText: "", totalScore: 0, accuracyScore: 0, flu… 0, 0, xml);
        mainHandler.post(() -> {
            if (callback != null) callback.onResult(fallback);
        });
    }
}
```

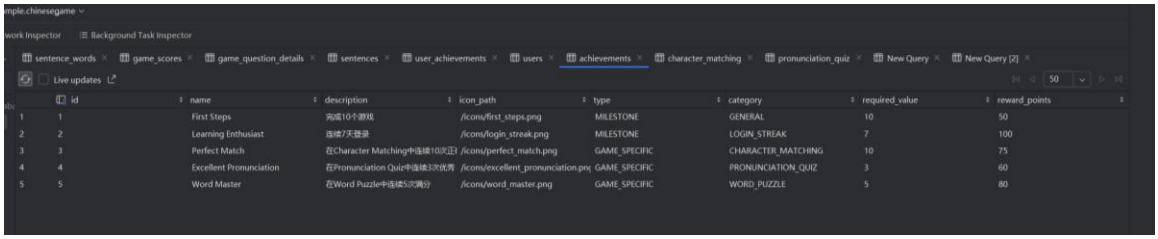
7. Reward and Achievement System

A basic reward system has been implemented to motivate user engagement:

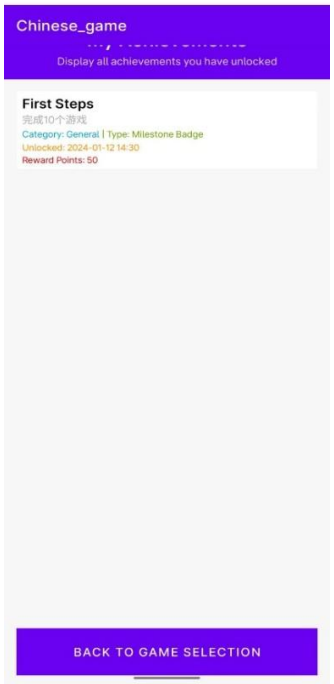
- Achievement Definitions: Loaded from **achievements.json**, including:
  a) Milestone Badges — cumulative targets (e.g., "First Steps" after 10 games, "Login Streak" for 7 consecutive days)
  b) Game-Specific Badges — conditional achievements (e.g., "Perfect Match" for 10-streak in Character Matching, "Excellent Pronunciation" for 3-streak in Pronunciation Quiz)
- Achievement Tracking: **AchievementDao** implements **checkAndUnlockAchievements()** to evaluate unlock conditions after each game session

- User Interface: **UserAchievementsActivity** displays all earned achievements in a scrollable list
- Score Persistence: **GameScoreDao** records session scores; **GameQuestionDetailDao** tracks per-question performance for detailed analytics

Achievement table



id	name	description	icon_path	type	category	required_value	reward_points
1	First Steps	完成10个游戏	/icons/first_steps.png	MILESTONE	GENERAL	10	50
2	Learning Enthusiast	连续7天登录	/icons/login_streak.png	MILESTONE	LOGIN_STREAK	7	100
3	Perfect Match	在Character Matching中连续10次正	/icons/perfect_match.png	GAME_SPECIFIC	CHARACTER_MATCHING	10	75
4	Excellent Pronunciation	在Pronunciation Quiz中连续3次优秀	/icons/excellent_pronunciation.pn	GAME_SPECIFIC	PRONUNCIATION_QUIZ	3	60
5	Word Master	在Word Puzzle中连续5次满分	/icons/word_master.png	GAME_SPECIFIC	WORD_PUZZLE	5	80



尚需完成的任务 Work to do:

1) Word Puzzle Game Implementation
   Utilize the pre-processed segmentation results from HanLP to implement the core sentence rearrangement gameplay of the Word Puzzle game, apply a dedicated shuffling algorithm to randomly disrupt the word order for each individual puzzle, and since the relevant DAO (WordPuzzleDao), data model (WordPuzzle), and JSON data file (word_puzzle.json) have all been pre-prepared, focus on developing the intuitive drag-and-drop or tap-to-reorder interaction to enable users to arrange words and complete the puzzle.
2) Achievement System Enhancement
   Develop and implement the complete unlock logic for all types of achievement badges and add real-time notification popups to prompt users when an achievement is unlocked, add intuitive visual progress indicators to display the completion status of partially finished achievements for users' reference, and further connect the achievement check mechanism with all game types in the app, optimizing the currently basic connection logic to ensure comprehensive achievement tracking across gameplay.
3) UI/UX Polish and Enhancement
   Upgrade the app's visual design by adding smooth animations and transition effects between different screens to improve the fluency of page switching, create dedicated game result

summary screens that display detailed score breakdowns for users to review their game performance, enhance the game selection interface by adding visual previews for each game type to help users quickly understand gameplay features, and optimize the overall layout to achieve adaptive display for different Android device screen sizes and orientations, ensuring a consistent user experience across devices.

4) Comprehensive Testing
Write professional unit tests to verify the functional correctness of all DAO classes and the core game logic of the app, create detailed instrumented tests to conduct end-to-end verification for all complete UI interaction flows, carry out comprehensive compatibility testing by running the app across different Android devices and various API levels to identify and fix adaptation issues, and perform targeted performance testing for key processes including database queries and speech SDK initialization to test their running efficiency and stability.

5) Performance Optimization
Optimize the loading efficiency of JSON data files, with a key focus on reducing the loading time of the large sentences.json file containing 3794 lines of data, optimize the overall database query performance by adding proper and reasonable indexes to relevant database tables to speed up data retrieval, and conduct technical optimization for the speech SDK to effectively reduce its initialization latency and improve the app's startup and functional response speed.

6) Final Report Writing
Systematically document the entire development process of the project, including the background, key technical decisions made at each stage, and the detailed rationale for the platform migration from Unity to native Android development, comprehensively incorporate all evaluation results and analysis data obtained from the various testing phases of the project into the report to reflect the app's performance and usability, and complete the writing of all required chapters in strict accordance with the pre-defined final report structure to ensure the report is complete, logical and standardized.

存在问题 Problems:

1) HanLP Library Size on Mobile Devices
Running a full NLP library on mobile devices presented memory and performance challenges. The standard HanLP distribution includes large dictionary files that could impact app size and loading time.

2) Speech Recognition Accuracy for Non-native Speakers
The iFlyTek ISE engine is primarily trained on native Chinese speakers. Non-native learners with varying accents may receive lower scores, which could affect user motivation and the fairness of the scoring system.

3) Unreasonable Difficulty Classification of Questions Across Three Game Modes
The current difficulty grading for questions in the Character-Pinyin Matching, Pronunciation Quiz and upcoming Word Puzzle games is based on subjective manual labeling, lacking a unified and data-driven difficulty evaluation standard. This leads to inconsistent difficulty calibration between different game modes—for example, some Medium-level questions in the Pronunciation Quiz are actually equivalent to Hard-level in the Character-Pinyin Matching game. Meanwhile, individual game modes have internal difficulty confusion, such as overly simple Hard-level questions or overly challenging Easy-level questions in partial game sections, which seriously affects the user's learning and gaming experience and fails to achieve the intended progressive difficulty guidance for learners.

拟采取的办法 Solutions:

1) HanLP Optimization
   Adopted the portable version (1.8.6) of HanLP, which is optimized for smaller footprints. Pre-processed segmentation results during data loading via SeedDataLoader and cached results in the SQLite database, so HanLP does not need to run during gameplay.

2) Pronunciation Scoring Normalization
   Mapped raw ISE scores (0–100) to a normalized game score (0–10) with learner-friendly thresholds. Provided multi-dimensional feedback (accuracy, fluency, completeness, tone) so users can understand specific areas for improvement rather than just seeing a single number.

3) Standardized Difficulty Classification System for Cross-Game Questions
   First, establish a unified multi-dimensional difficulty evaluation index system applicable to all three game modes, including core indicators such as Chinese character stroke count, word frequency of vocabulary, pinyin/tone complexity, sentence length and syntactic difficulty, and assign scientific weight coefficients to each indicator according to the characteristics of different gameplays. Second, use a data-driven labeling method to replace manual subjective labeling: based on the established index system, write an automated scoring script to calculate the difficulty score of each question, and divide the questions into Easy, Medium and Hard levels with clear score thresholds to ensure cross-game difficulty consistency.

论文结构 Structure of the final report: (Chapter headings and section sub headings)

Chapter 1: Introduction
- 1.1 Background and Motivation
- 1.2 Project Aims and Objectives
- 1.3 Report Structure

Chapter 2: Literature Review
- 2.1 Gamification in Language Learning
- 2.2 Review of Existing Chinese Learning Applications
  - 2.2.1 Duolingo
  - 2.2.2 Baicizhan
  - 2.2.3 Other Chinese Learning Tools
- 2.3 Speech Recognition and Evaluation Technologies
- 2.4 Natural Language Processing for Chinese
- 2.5 Mobile Application Development Frameworks
- 2.6 Summary

Chapter 3: Requirements Analysis
- 3.1 User Requirements
- 3.2 Functional Requirements
  - 3.2.1 User Management
  - 3.2.2 Game System
  - 3.2.3 Reward System
- 3.3 Non-functional Requirements
- 3.4 Use Case Diagrams
- 3.5 Ethical Considerations

Chapter 4: System Design
- 4.1 System Architecture Overview
- 4.2 Database Design
  - 4.2.1 Entity-Relationship Diagram
  - 4.2.2 Table Schema Definitions
- 4.3 UI/UX Design

4.3.1 Wireframes
4.3.2 Navigation Flow
4.4 Game Mechanics Design
4.4.1 Character-Pinyin Matching
4.4.2 Pronunciation Quiz
4.4.3 Word Puzzle
4.5 API and SDK Integration Design
4.5.1 iFlyTek ISE Integration
4.5.2 HanLP and pinyin4j Integration
4.6 Difficulty and Reward System Design

Chapter 5: Implementation
5.1 Development Environment and Tools
5.2 Technology Selection and Platform Migration
5.3 Database Implementation
5.4 User Authentication and Profile Management
5.5 Character-Pinyin Matching Game Implementation
5.6 Pronunciation Quiz Implementation
5.6.1 IseEvaluator WebSocket Architecture
5.6.2 Audio Recording and Streaming
5.6.3 Score Parsing and Normalization
5.7 Word Puzzle Implementation
5.7.1 HanLP Sentence Segmentation
5.7.2 Shuffling Algorithm
5.8 Achievement and Reward System
5.9 Data Loading and Management

Chapter 6: Testing and Evaluation
6.1 Testing Strategy
6.2 Unit Testing
6.3 Integration Testing
6.4 UI/UX Testing
6.5 Performance Testing
6.6 Cross-device Compatibility Testing
6.7 User Acceptance Testing
6.8 Test Results Summary

Chapter 7: Conclusion
7.1 Summary of Achievements
7.2 Limitations
7.3 Future Work

References

Appendices
Appendix A: Full Database Schema
Appendix B: Key Code Listings
Appendix C: User Guide
Appendix D: Test Results